

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
BỘ MÔN MÁY TÍNH - HỆ THỐNG NHÚNG



NGUYỄN MINH TRÍ

Đề tài đồ án tốt nghiệp:

**HỆ THỐNG ĐĂNG KÝ VÀ NHẬN DIỆN BIỂN
SỐ XE TỰ ĐỘNG**

Chuyên ngành Máy Tính – Hệ Thống Nhúng

TP. Hồ Chí Minh, tháng 7 năm 2025

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
BỘ MÔN MÁY TÍNH - HỆ THỐNG NHÚNG



NGUYỄN MINH TRÍ

21200363

Đề tài:

**HỆ THỐNG NHẬN DIỆN ĐĂNG KÝ BIỂN SỐ XE
TỰ ĐỘNG**

**AUTOMATIC VEHICLE LICENSE PLATE RECOGNITION
AND REGISTRATION SYSTEM**

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG
CHUYÊN NGÀNH MÁY TÍNH - HỆ THỐNG NHÚNG

NGƯỜI HƯỚNG DẪN KHOA HỌC
ThS. NGUYỄN QUỐC KHOA

TP. Hồ Chí Minh, tháng 7 năm 2025

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài “Hệ thống nhận diện đăng ký biển số xe tự động”, em đã nhận được rất nhiều sự hỗ trợ từ quý thầy cô, gia đình và bạn bè. Em xin bày tỏ lòng biết ơn sâu sắc đến tất cả những ai đã đồng hành và tạo điều kiện để em hoàn thành đề tài này.

Trước tiên, em xin gửi lời cảm ơn chân thành đến quý thầy cô Khoa Điện tử – Viễn thông, Trường Đại học Khoa học Tự nhiên, những người đã truyền đạt kiến thức và khơi gợi niềm đam mê nghiên cứu trong suốt những năm học. Đặc biệt, em biết ơn các thầy cô bộ môn Máy tính hệ thống nhúng, đã trang bị cho em nền tảng kiến thức chuyên sâu về lập trình nhúng, xử lý dữ liệu và tư duy hệ thống, giúp em triển khai đề tài một cách thực tiễn và hiệu quả.

Em đặc biệt cảm ơn Thầy Nguyễn Quốc Khoa – giảng viên hướng dẫn, người đã tận tâm định hướng, góp ý và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những chia sẻ quý báu của thầy không chỉ giúp em hoàn thiện đề tài mà còn là động lực để em tiếp tục học hỏi và phát triển.

Dù đã nỗ lực, em hiểu rằng đề tài vẫn còn nhiều hạn chế. Em rất mong nhận được góp ý từ quý thầy cô và các bạn để hoàn thiện hơn nữa sản phẩm.

Cuối cùng, em xin kính chúc quý thầy cô trong Khoa Điện tử – Viễn thông luôn mạnh khỏe và tiếp tục là nguồn cảm hứng cho các thế hệ sinh viên.

Trân trọng
Nguyễn Minh Trí

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong đồ án tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định. Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

ký tên

Họ tên sinh viên

TÓM TẮT ĐỒ ÁN TỐT NGHIỆP

Lý do chọn đề tài:

Cùng với sự phát triển của nền kinh tế và chất lượng đời sống, nhu cầu tham gia giao thông ngày càng tăng cao, kéo theo sự gia tăng nhanh chóng về số lượng phương tiện, đặc biệt là xe máy. Điều này đặt ra yêu cầu cấp thiết về một hệ thống quản lý phương tiện hiện đại và hiệu quả. Một trong những giải pháp được quan tâm hiện nay là ứng dụng công nghệ vào việc đăng ký và nhận diện biển số xe tự động. Hệ thống này không chỉ giúp nhận diện phương tiện chính xác mà còn hỗ trợ công tác đăng ký, theo dõi và quản lý thông tin biển số một cách thuận tiện.

Tóm tắt nội dung:

Cũng như các hệ thống nhận dạng khác, hệ thống bao gồm hai phần chính: phần cứng và phần mềm. Phần cứng sử dụng ESP32 CAM để thu nhận hình ảnh biển số xe. Phần mềm sẽ xử lý và phân tích hình ảnh để trích xuất ký tự trên biển số, kết hợp với cơ sở dữ liệu để thực hiện chức năng đăng ký và nhận diện:

- Quy trình đăng ký biển số: Người dùng (hoặc quản trị viên) nhập thông tin xe và tải hình ảnh biển số để hệ thống lưu vào cơ sở dữ liệu.
- Quy trình nhận diện biển số: Hệ thống tự động nhận diện biển số từ hình ảnh thực tế và so sánh với dữ liệu đã đăng ký để xác minh.

Chi tiết các bước:

- Hệ thống sử dụng mô hình YOLO để phát hiện và khoanh vùng khu vực chứa biển số xe dựa trên đặc trưng hình học và dữ liệu huấn luyện. Sau khi cắt ra vùng biển số, ảnh sẽ được chuyển sang định dạng ảnh xám và tiếp tục xử lý bằng các kỹ thuật như làm mịn, cân bằng sáng, và nhị phân hóa ảnh để thuận tiện cho các bước nhận dạng tiếp theo.
- Phân đoạn ký tự: Sau khi chuẩn hóa vùng ảnh biển số, hệ thống sử dụng các kỹ thuật xử lý ảnh truyền thống như phân tích biên (contour detection) hoặc projection profile để tách riêng từng ký tự.
- Nhận diện ký tự: Ký tự được nhận diện bằng các phương pháp so khớp mẫu hoặc phân tích đặc trưng hình dạng dựa trên hình ảnh nhị phân của từng ký

tự. Quá trình nhận dạng này dựa trên đối sánh trực tiếp với tập dữ liệu mẫu đã chuẩn bị trước.

Mục tiêu:

- Xây dựng hệ thống nhận diện và đăng ký biển số xe tự động dựa trên xử lý ảnh.
- Tích hợp mô hình YOLO để phát hiện chính xác biển số từ ảnh thực tế.
- Thiết kế giao diện đăng ký, lưu trữ và tra cứu thông tin biển số.
- Tối ưu hóa quá trình xử lý ảnh nhằm tăng độ chính xác trong điều kiện thực tế.

Giới hạn đề tài:

- Hệ thống chỉ hỗ trợ nhận diện các biển số có định dạng chuẩn: nền trắng, chữ đen, dạng một hoặc hai hàng, mỗi hàng gồm 4 đến 5 ký tự, tổng cộng từ 8 đến 10 ký tự.
- Hình ảnh đầu vào cần đảm bảo độ rõ nét tối thiểu, không bị mờ, biến dạng nghiêm trọng, và các ký tự phải đủ rõ để có thể phân biệt bằng mắt thường.
- Hệ thống không xử lý được các trường hợp ảnh bị lóa, chói sáng mạnh hoặc biển số bị che khuất một phần hoặc toàn bộ.
- Góc chụp biển số không được nghiêng quá mức (ví dụ: quá nghiêng theo phương ngang hoặc dọc), vì điều này làm giảm khả năng phát hiện và nhận diện chính xác của mô hình

MỤC LỤC

DANH SÁCH TỪ VIẾT TẮT	7
DANH SÁCH HÌNH	8
DANH SÁCH BẢNG	9
LỜI MỞ ĐẦU.....	10
Phần 1: CƠ SỞ LÝ THUYẾT	11
1.1 GIỚI THIỆU VỀ HỆ THỐNG NHẬN DIỆN BIÊN SỐ.	11
1.1.1 Thu thập dữ liệu.....	12
1.1.2 Xử lý hình ảnh	12
1.1.3 Tách vùng biên số.....	12
1.1.4 Huấn luyện mô hình.....	12
1.1.5 Triển khai mô hình.....	13
1.1.6 Tối ưu hóa	13
1.2 Cấu trúc hệ thống.....	13
1.2.1 Nền tảng phần cứng	13
1.2.2 Phần mềm và công cụ sử dụng	15
Phần 2: THỰC NGHIỆM.....	17
2.1 QUÁ TRÌNH HUẤN LUYỆN MÔ HÌNH.....	17
2.1.1 Tổng quan về YOLO	17
2.1.2 Chi tiết về mô hình sử dụng: YOLOv5	17
2.1.3 So sánh các phiên bản.....	19
2.1.4 Thành phần trong bài toán	20
2.2 Quá trình huấn luyện	21
2.2.1 Bộ dữ liệu.....	21

2.2.2	Các chỉ số đánh giá hiệu suất mô hình	23
2.2.3	Kết quả mô hình nhận diện biên số	26
2.2.4	Kết quả mô hình nhận diện ký tự.....	28
2.3	TÍCH HỢP MÔ HÌNH VÀO HỆ THỐNG	31
2.3.1	Mô tả tổng quan	31
2.3.2	Kết nối với ESP32-CAM.....	32
2.3.3	Tài khoản người dùng.....	35
2.3.4	Tài khoản quản trị viên	35
2.3.5	Cơ sở dữ liệu.....	36
Phần 3: KẾT QUẢ VÀ ĐÁNH GIÁ		40
3.1	Kết quả thực tế.....	40
3.1.1	Nhận diện hình ảnh	40
3.1.2	Hoạt động hệ thống.....	41
3.2	Đánh giá và nhận xét	41
3.2.1	Tính ứng dụng thực tiễn.....	41
3.2.2	Đề xuất hướng cải tiến.....	42
TÀI LIỆU THAM KHẢO		44

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ
SVM	Support Vector Machine – thuật toán máy học dùng để phân loại dữ liệu.
KNN	K-Nearest Neighbors – thuật toán phân loại dựa trên các điểm dữ liệu gần nhất.
YOLO	You Only Look Once – mô hình nhận diện vật thể.
mAP	Mean Average Precision – độ chính xác trung bình của mô hình.
JPEG	Joint Photographic Experts Group – Định dạng hình ảnh phổ biến, với kích cỡ dung lượng ảnh nhỏ.
MJPEG	Motion Joint Photographic Experts Group – Một chuỗi ảnh JPEG liên tiếp tạo thành một video.
ERD	Entity Relationship Diagram – Sơ đồ mô tả mối quan hệ giữa các thực thể trong cơ sở dữ liệu.
OCR	Optical Character Recognition – Công nghệ nhận dạng và chuyển đổi văn bản từ hình ảnh sang dạng ký tự.
ONNX	Open Neural Network Exchange – Định dạng mở để chuyển đổi mô hình AI giữa các framework khác nhau.
FTDI	Future Technology Devices International - là một công ty thiết bị bán dẫn tư nhân của Scotland, chuyên về công nghệ Universal Serial Bus

DANH SÁCH HÌNH

<i>Hình 1.1 Sơ đồ chân của ESP32-CAM</i>	<i>15</i>
<i>Hình 2.1 Tổng quan quá trình với YOLO</i>	<i>17</i>
<i>Hình 2.2 Cấu trúc của mô hình YOLO.....</i>	<i>18</i>
<i>Hình 2.3 So sánh các phiên bản YOLO</i>	<i>19</i>
<i>Hình 2.4 Hình ảnh ví dụ cho quá trình đánh nhãn</i>	<i>22</i>
<i>Hình 2.5 Ma trận nhầm lẫn sau khi huấn luyện mô hình.....</i>	<i>24</i>
<i>Hình 2.6 Kết quả nhận diện các ký tự xuất hiện trong bộ dữ liệu</i>	<i>25</i>
<i>Hình 2.7 Sơ đồ hoạt động của mô hình.....</i>	<i>26</i>
<i>Hình 2.8 Các đồ thị trong quá trình huấn luyện mô hình nhận diện biển số</i>	<i>27</i>
<i>Hình 2.9 Kết quả huấn luyện mô hình nhận diện biển số</i>	<i>28</i>
<i>Hình 2.10 Các đồ thị trong quá trình huấn luyện mô hình nhận diện ký tự</i>	<i>29</i>
<i>Hình 2.11 Kết quả huấn luyện mô hình nhận diện ký tự.....</i>	<i>30</i>
<i>Hình 2.12 Giao diện đăng nhập chính của hệ thống</i>	<i>31</i>
<i>Hình 2.13 Sơ đồ sơ lược quy trình hoạt động của hệ thống</i>	<i>32</i>
<i>Hình 2.14 Sơ đồ kết nối ESP32-CAM với mạch chuyển FTDI</i>	<i>35</i>
<i>Hình 2.15 Giao diện chính của người dùng.....</i>	<i>35</i>
<i>Hình 2.16 Giao diện chính của quản trị viên.....</i>	<i>36</i>
<i>Hình 2.17 Mô hình ERD của cơ sở dữ liệu</i>	<i>36</i>

DANH SÁCH BẢNG

Bảng 1.1 Bảng liệt kê chân pin	15
Bảng 2.1 Nội dung của file txt	22
Bảng 2.2 Đoạn lệnh huấn luyện mô hình nhận diện biến số.....	26
Bảng 2.3 Tóm tắt kết quả quá trình huấn luyện mô hình nhận diện biến số.....	28
Bảng 2.4 Đoạn lệnh huấn luyện mô hình nhận diện ký tự.....	28
Bảng 2.5 Tóm tắt kết quả quá trình huấn luyện mô hình nhận diện ký tự.....	31
Bảng 2.6 Lớp dữ liệu của người dùng.....	37
Bảng 2.7 Lớp dữ liệu của đơn đăng ký	37
Bảng 2.8 Lớp dữ liệu của quản trị viên.....	38

LỜI MỞ ĐẦU

Trong những năm gần đây, cùng với sự phát triển của đô thị hóa và phương tiện cá nhân, nhu cầu xây dựng các hệ thống quản lý phương tiện thông minh ngày càng trở nên cấp thiết. Một trong những giải pháp hiệu quả là ứng dụng công nghệ xử lý ảnh vào việc tự động nhận diện và quản lý biển số xe. Đề tài “Hệ thống đăng ký và nhận diện biển số xe tự động” được thực hiện nhằm đáp ứng yêu cầu đó.

Hệ thống bao gồm hai thành phần chính: phần phần cứng là camera hoặc webcam dùng để ghi nhận hình ảnh biển số, và phần phần mềm đảm nhận xử lý ảnh và nhận diện ký tự. Sau khi thu nhận ảnh, hệ thống tiến hành xử lý để tách vùng chứa biển số, phân đoạn ký tự, và cuối cùng trích xuất nội dung biển số. Biển số sau khi nhận diện sẽ được so sánh với cơ sở dữ liệu đăng ký để xác minh hoặc ghi nhận thông tin phương tiện.

Điểm nổi bật của hệ thống là không sử dụng các thuật toán học sâu mà chỉ áp dụng các kỹ thuật xử lý ảnh cơ bản như làm mờ, chuyển đổi ảnh xám, nhị phân hóa, phát hiện cạnh (Canny), tìm hình chữ nhật tương đồng với tỉ lệ biển số, và so khớp mẫu ký tự theo phương pháp thống kê hoặc theo mẫu ảnh chuẩn. Nhờ vậy, hệ thống có thể triển khai với chi phí thấp, dễ dàng tùy chỉnh, và phù hợp với nhiều điều kiện thực tế.

Thông qua đề tài này, người thực hiện mong muốn xây dựng một mô hình hệ thống có tính ứng dụng cao trong thực tiễn, góp phần hỗ trợ công tác quản lý phương tiện tại các khu dân cư, cơ quan, bãi đỗ xe hoặc các trạm kiểm soát an ninh.

Phần 1: CƠ SỞ LÝ THUYẾT

1.1 GIỚI THIỆU VỀ HỆ THỐNG NHẬN DIỆN BIỂN SỐ.

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ Machine Learning đã mở ra nhiều hướng tiếp cận mới trong lĩnh vực xử lý ảnh và nhận diện đối tượng.

Thay vì chỉ dựa vào các phương pháp xử lý ảnh truyền thống, hiện nay việc sử dụng các mô hình học máy để “học” từ dữ liệu ảnh và tự động đưa ra kết quả phân loại đang ngày càng phổ biến và hiệu quả. Mô hình học máy hoạt động dựa trên nguyên tắc: từ một tập dữ liệu huấn luyện lớn (training dataset), thuật toán sẽ tự động tìm ra các mẫu (patterns) và mối quan hệ giữa đầu vào (hình ảnh) và đầu ra (trong đề tài này là các ký tự xuất hiện trên biển số xe).

Sau quá trình huấn luyện, mô hình có thể dự đoán chính xác đầu ra cho các ảnh mới mà chưa từng gặp trước đó.

Quy trình xây dựng mô hình nhận diện biển số xe thường bao gồm các bước chính sau:

- **Thu thập dữ liệu:** Xây dựng bộ dữ liệu ảnh biển số xe với nhiều tình huống và điều kiện khác nhau (góc chụp, ánh sáng, chất lượng ảnh...).
- **Tiền xử lý dữ liệu:** Làm sạch dữ liệu, chuẩn hóa kích thước ảnh, điều chỉnh độ tương phản và loại bỏ nhiễu để giúp mô hình học hiệu quả hơn, sau đó đánh nhãn mỗi bức ảnh bằng các file .txt.
- **Huấn luyện mô hình:** Sử dụng các thuật toán học máy để huấn luyện mô hình từ dữ liệu ảnh đã xử lý.
- Các thuật toán phổ biến gồm: SVM, KNN, hoặc các mô hình học sâu (deep learning) khi yêu cầu độ chính xác cao.
- **Đánh giá mô hình:** Kiểm tra độ chính xác của mô hình trên tập dữ liệu kiểm thử (validation/test dataset), tinh chỉnh các siêu tham số (hyperparameters) nếu cần để tối ưu kết quả.
- **Triển khai:** Tích hợp mô hình vào hệ thống thực tế, cho phép nhận diện biển số xe từ ảnh đầu vào một cách tự động.

Việc sử dụng machine learning giúp cho hệ thống nhận diện biển số xe có thể thích ứng với nhiều loại biển số khác nhau và xử lý tốt hơn các biến động trong

môi trường thực tế (ánh sáng, góc chụp, chất lượng camera). So với các phương pháp cứng dựa vào quy tắc thủ công, mô hình học máy có khả năng cải thiện độ chính xác qua thời gian, khi được cung cấp thêm dữ liệu huấn luyện mới.

1.1.1 Thu thập dữ liệu

Đây là bước đầu tiên và rất quan trọng, bởi chất lượng của tập dữ liệu quyết định phần lớn hiệu quả của mô hình. Một tập hợp ảnh biển số xe đa dạng cần được thu thập, bao gồm nhiều điều kiện môi trường khác nhau như ánh sáng mạnh hoặc yếu, ngày và đêm, các góc chụp nghiêng, chất lượng ảnh từ các loại camera khác nhau. Mỗi ảnh phải được gán nhãn chính xác với thông tin biển số tương ứng, tạo cơ sở cho quá trình huấn luyện.

1.1.2 Xử lý hình ảnh

Sau khi thu thập, tập ảnh sẽ được xử lý sơ bộ để chuẩn hóa đầu vào. Quá trình này bao gồm chuyển đổi định dạng ảnh về chuẩn phù hợp, resize ảnh về cùng kích thước cố định, điều chỉnh độ sáng, độ tương phản và khử nhiễu nếu cần thiết. Ngoài ra, để giúp mô hình có khả năng tổng quát tốt hơn, các kỹ thuật tăng cường dữ liệu như xoay ảnh nhẹ, thay đổi độ sáng hoặc lật ảnh sẽ được áp dụng. Điều này giúp mô hình học được nhiều dạng biến thể khác nhau của biển số xe ngoài đời thực.

1.1.3 Tách vùng biển số

Ở bước này, hệ thống sẽ phát hiện và tách riêng vùng chứa biển số từ ảnh toàn cảnh. Tùy theo phương pháp tiếp cận, có thể sử dụng các kỹ thuật truyền thống như Haar cascade hoặc HOG + SVM, hoặc các thuật toán hiện đại hơn như YOLO hoặc SSD để phát hiện nhanh và chính xác vùng biển số. Phần ảnh biển số sau khi được cắt ra sẽ là đầu vào cho các bước nhận diện tiếp theo.

Khi vùng biển số đã được tách riêng, bước tiếp theo là phân đoạn từng ký tự trên biển số. Đây là bước tách riêng từng ký tự để mô hình có thể nhận diện từng phần của biển số. Các kỹ thuật thường dùng gồm phát hiện đường biên, phân ngưỡng ảnh và phân vùng kết nối. Kết quả là từng ký tự riêng biệt được chuẩn hóa kích thước, sẵn sàng cho quá trình nhận diện.

1.1.4 Huấn luyện mô hình

Mô hình học máy sẽ được thiết kế và huấn luyện trên tập dữ liệu ký tự đã qua xử lý. Tùy theo yêu cầu và độ phức tạp, có thể sử dụng các thuật toán học máy

truyền thống như SVM hoặc KNN, hoặc các mô hình sâu hơn như CNN nếu cần độ chính xác cao. Tập dữ liệu sẽ được chia thành các phần: tập huấn luyện, tập xác thực và tập kiểm thử. Trong quá trình huấn luyện, các tham số của mô hình sẽ được điều chỉnh liên tục để tối ưu hiệu năng.

Sau khi huấn luyện, mô hình sẽ được đánh giá trên tập kiểm thử độc lập nhằm kiểm tra khả năng tổng quát và độ chính xác. Các chỉ số như accuracy, precision, recall và F1-score sẽ được dùng để đo lường chất lượng mô hình. Việc phân tích confusion matrix cũng giúp xác định các trường hợp nhận diện sai để có thể cải thiện ở các vòng huấn luyện sau.

1.1.5 Triển khai mô hình

Mô hình sau khi đạt hiệu năng mong muốn sẽ được đóng gói và tích hợp vào hệ thống thực tế. Tùy thuộc vào nền tảng, mô hình có thể được triển khai trên các hệ thống máy tính, máy chủ web hoặc trên các thiết bị nhúng có phần cứng giới hạn. Việc tối ưu hóa mô hình để có thể chạy được thời gian thực là yêu cầu quan trọng trong giai đoạn này.

1.1.6 Tối ưu hóa

Sau khi triển khai, hệ thống cần tiếp tục được duy trì và cập nhật. Các trường hợp nhận diện sai sẽ được ghi nhận để bổ sung vào tập dữ liệu huấn luyện. Qua các vòng huấn luyện lại định kỳ, mô hình sẽ ngày càng cải thiện khả năng nhận diện và đáp ứng tốt hơn với các tình huống ngoài thực tế.

1.2 Cấu trúc hệ thống

1.2.1 Nền tảng phần cứng

ESP32-CAM là một module tích hợp vi điều khiển ESP32 với khả năng kết nối Wi-Fi và Bluetooth, đồng thời có sẵn giao diện camera OV2640, cho phép chụp ảnh và truyền dữ liệu hình ảnh qua mạng. Trong project này, ESP32-CAM được sử dụng để thu nhận ảnh biển số xe theo thời gian thực, làm đầu vào cho hệ thống xử lý nhận diện.

Thông số kỹ thuật chính của ESP32-CAM [1]:

- Vi xử lý: ESP32-D0WD, dual-core Xtensa® 32-bit LX6, tốc độ 240 MHz
- RAM: 520 KB SRAM tích hợp, mở rộng thêm PSRAM 4 MB
- Bộ nhớ Flash: 4 MB

- Kết nối: Wi-Fi 802.11 b/g/n, Bluetooth v4.2 (BR/EDR và BLE)
- Camera: OV2640 2 Megapixel CMOS sensor
- Độ phân giải tối đa: 1600x1200 (UXGA)
- Độ phân giải hỗ trợ: từ QQVGA (160x120) đến UXGA
- Hỗ trợ JPEG, BMP Giao tiếp: GPIO, UART, SPI, I2C, PWM
- Nguồn cấp: 5V DC, tiêu thụ trung bình khoảng 160-180 mA khi sử dụng camera
- Kích thước: 27 mm x 40.5 mm Thẻ nhớ: hỗ trợ MicroSD (TF card) lên đến 32 GB để lưu ảnh cục bộ

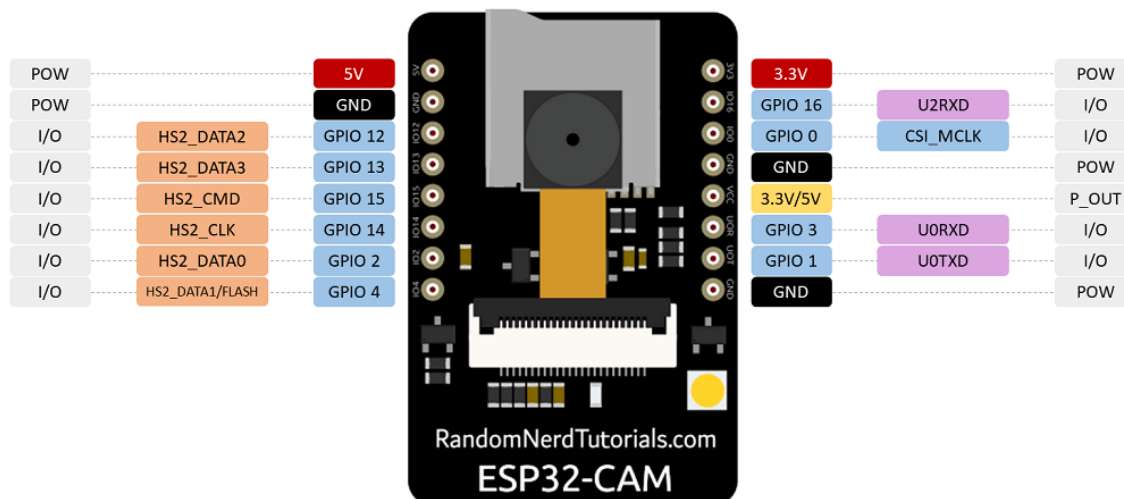
Ưu điểm của ESP32-CAM:

- Chi phí thấp, dễ tiếp cận cho các ứng dụng IoT nhỏ và trung bình.
- Kích thước nhỏ gọn, dễ tích hợp vào các hệ thống nhúng hoặc thiết bị di động.
- Kết nối Wi-Fi mạnh mẽ, có thể truyền hình ảnh trực tiếp tới server hoặc trình duyệt.
- Khả năng lập trình linh hoạt, có thể sử dụng với Arduino IDE, PlatformIO hoặc ESP-IDF.
- Hỗ trợ chụp ảnh, quay video (frame capture) và lưu ảnh vào thẻ nhớ khi cần.

Nhược điểm của ESP32-CAM:

- Không có cổng USB tích hợp, cần sử dụng USB-to-TTL (UART) rời để nạp chương trình.
- Dung lượng RAM và tài nguyên xử lý hạn chế khi chạy các tác vụ xử lý ảnh phức tạp; phù hợp chủ yếu cho tác vụ thu nhận và truyền ảnh, chứ không lý tưởng để xử lý ảnh AI trực tiếp trên board.
- Độ trễ livestream khoảng 100-300ms, tùy theo độ phân giải và tốc độ mạng Wi-Fi.
- Chất lượng hình ảnh phụ thuộc nhiều vào ánh sáng môi trường; trong điều kiện thiếu sáng, ảnh có thể nhiễu.

Trong phạm vi của đề tài này, ESP32-CAM đóng vai trò là thiết bị thu nhận ảnh đầu vào cho hệ thống nhận diện biển số xe, thực hiện chức năng livestream thu nhận hình ảnh thời gian thực tới server hoặc máy tính xử lý. Với chi phí thấp và khả năng truyền ảnh linh hoạt, ESP32-CAM là lựa chọn phù hợp cho các ứng dụng giám sát giao thông hoặc quản lý phương tiện quy mô nhỏ và vừa [2].



Hình 1.1 Sơ đồ chân của ESP32-CAM

Bảng 1.1 Bảng liệt kê chân pin

STT	Tên Chân	Ghi Chú
1	5V	Nguồn vào 5V
3	GPIO 12	HS2_DATA2
4	GPIO 13	HS2_DATA3
5	GPIO 15	HS2_CMD
6	GPIO 14	HS2_CLK
7	GPIO 2	HS2_DATA0
8	GPIO 4	HS2_DATA1 / FLASH
9	3.3V	Nguồn vào 3.3V
10	GPIO 16	U2RXD
11	GPIO 0	CSI_MCLK / BOOT
13	3.3V / 5V	Nguồn vào 3.3/5V
14	GPIO 3	U0RXD
15	GPIO 1	U0TXD

1.2.2 Phần mềm và công cụ sử dụng

Về phần mềm, hệ thống được xây dựng dựa trên sự kết hợp của ba nền tảng chính: lập trình nhúng với ngôn ngữ C/C++ cho ESP32-CAM, Python cho xử lý ảnh và các thành phần backend của website. Ngoài ra, về phần dữ liệu sẽ sử dụng MySQL để tạo tài khoản người dùng/quản trị viên cũng như lưu trữ dữ liệu biển số, các đơn đăng ký của người dùng.

Trên module ESP32-CAM, chương trình được phát triển bằng ngôn ngữ C/C++ sử dụng Arduino framework, nhờ đó dễ dàng lập trình, mở rộng và tương thích với các thư viện hỗ trợ camera và Wi-Fi. Các chức năng chính bao gồm: cấu hình kết nối Wi-Fi, khởi động camera OV2640, thu nhận ảnh liên tục và truyền dữ liệu ảnh đến server thông qua giao thức HTTP hoặc WebSocket.

Ngoài ra, ESP32 còn có thể chụp ảnh định kỳ hoặc khi có yêu cầu từ server và lưu ảnh xuống thẻ nhớ microSD nếu cần. Phần xử lý ảnh và nhận diện biển số xe được xây dựng bằng ngôn ngữ Python nhờ sự phong phú và mạnh mẽ của các thư viện hỗ trợ như OpenCV (xử lý ảnh), Pillow (xử lý ảnh cơ bản), kết hợp với các thư viện machine learning nếu cần như scikit-learn hoặc TensorFlow. Python cho phép xử lý linh hoạt ảnh đầu vào từ ESP32-CAM, bao gồm: đọc luồng ảnh, tách vùng biển số, phân đoạn ký tự và nhận diện nội dung biển số xe.

Phần website cũng được lập trình chủ yếu bằng Python, thông qua framework Flask để xây dựng các API và giao diện người dùng. Giao diện web cho phép quản trị viên theo dõi luồng ảnh livestream, kiểm tra kết quả nhận diện biển số, lưu trữ và truy vấn thông tin biển số trong cơ sở dữ liệu.

Kết hợp với đó là HTML/CSS và JavaScript để xây dựng phần giao diện hiển thị trên trình duyệt, phục vụ người dùng một cách thân thiện. Việc kết hợp giữa C/C++ (lập trình ESP32-CAM), Python (cho xử lý ảnh và backend website), cùng các công nghệ web phổ biến giúp hệ thống vừa đảm bảo hiệu suất cao trong phần thu nhận ảnh, vừa linh hoạt và dễ bảo trì trong các thành phần xử lý ảnh và giao diện người dùng.

Phần 2: THỰC NGHIỆM.

2.1 QUÁ TRÌNH HUẤN LUYỆN MÔ HÌNH

2.1.1 Tổng quan về YOLO

Đây là một trong những mô hình tiên tiến và phổ biến nhất trong lĩnh vực nhận diện và phát hiện đối tượng trong ảnh và video. Khác với các phương pháp truyền thống vốn dựa trên quá trình quét lặp nhiều vùng trong ảnh (region proposal), YOLO tiếp cận bài toán theo cách toàn cục: nó phân tích toàn bộ bức ảnh chỉ trong một lần xử lý duy nhất, từ đó dự đoán trực tiếp vị trí (bounding box) và nhãn phân loại của các đối tượng có mặt. Nhờ kiến trúc mạng nơ-ron tích chập tối ưu và khả năng xử lý song song, YOLO có thể đạt tốc độ xử lý rất cao, phù hợp cho các ứng dụng thời gian thực như giám sát an ninh, xe tự hành, hay robot thông minh.

Bên cạnh tốc độ, các phiên bản mới của YOLO (từ YOLOv3, YOLOv4, đến YOLOv8 hiện nay) còn cải thiện đáng kể độ chính xác và khả năng phát hiện nhiều đối tượng khác nhau trong những bối cảnh phức tạp. Nhờ sự cân bằng giữa hiệu năng và độ chính xác, YOLO đã trở thành một lựa chọn lý tưởng cho nhiều hệ thống thị giác máy tính hiện đại [3].

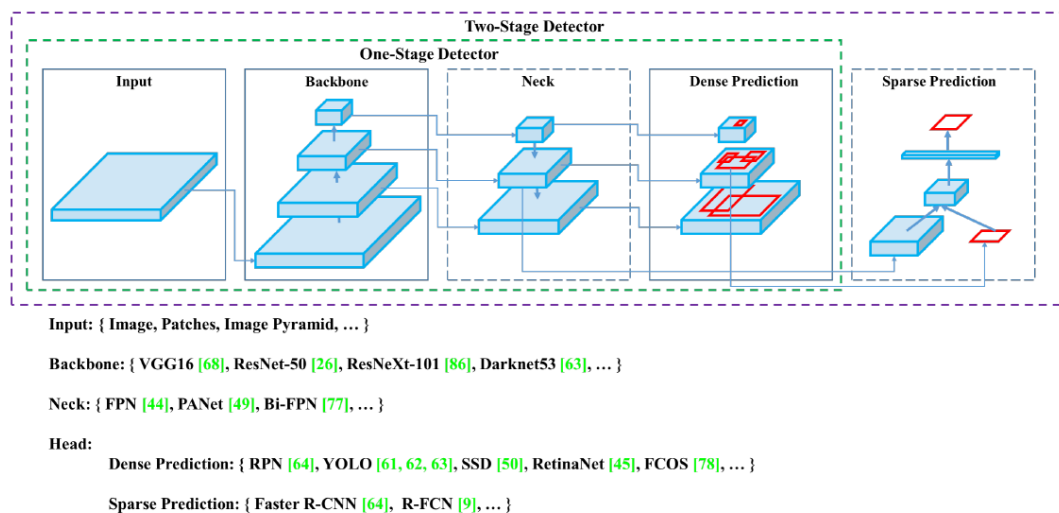


Hình 2.1 Tổng quan quá trình với YOLO

2.1.2 Chi tiết về mô hình sử dụng: YOLOv5

YOLOv5 là phiên bản thứ 5 của dòng mô hình YOLO, được phát triển bởi công ty Ultralytics, với mục tiêu phát hiện đối tượng trong hình ảnh và video. Khác với các phiên bản trước, YOLOv5 không có bài báo chính thức mà chỉ được phát hành dưới dạng mã nguồn mở trên GitHub. Tuy nhiên, YOLOv5 đã chứng minh

hiệu quả vượt trội về độ chính xác, tốc độ và hiệu suất so với YOLOv4. Bên cạnh đó, mô hình còn được tối ưu hóa để giảm kích thước, giúp dễ dàng triển khai trên các thiết bị tầm trung hoặc điện thoại thông minh [4].



Hình 2.2 Cấu trúc của mô hình YOLO

Kiến trúc YOLOv5 gồm ba thành phần chính:

- **Backbone:** Dựa trên kiến trúc CSPNet (Cross-Stage Partial Network), giúp cải thiện tốc độ và độ chính xác so với các phiên bản trước. CSPNet chia mạng CNN thành các tầng riêng biệt, mỗi tầng bao gồm phần tiền xử lý và hậu xử lý. Việc phân chia này giúp giảm độ phức tạp tính toán và tăng hiệu quả huấn luyện. Đây là phần đầu tiên của mô hình, có nhiệm vụ "nhìn" vào ảnh và trích xuất ra những đặc điểm quan trọng, giống như việc lọc ra các chi tiết nổi bật giúp phân biệt các đối tượng khác nhau.
- **Neck:** Sau khi có được các đặc điểm từ ảnh, phần này sẽ kết hợp và xử lý chúng để làm nổi bật thêm các thông tin cần thiết. Nhờ đó, mô hình có thể nhận diện tốt hơn các đối tượng dù chúng có kích thước lớn, nhỏ hay nằm ở vị trí khác nhau trong ảnh.
- **Head:** Được phát triển dựa trên kiến trúc của YOLOv3 nhưng đã được cải tiến và tối ưu để đạt độ chính xác và tốc độ cao hơn. Đây là phần thực hiện dự đoán cuối cùng — đưa ra khung bọc (bounding

box) quanh các đối tượng (người, đồ vật, vật thể...) và xác suất lớp cho các đối tượng đó trong ảnh.

2.1.3 So sánh các phiên bản

Model	Size (pixels)	mAPval 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	Params (M)	FLOPs (B)
YOLOv5n	640	34.3	73.6	1.06	2.6	7.7
YOLOv5s	640	43.0	120.7	1.27	9.1	24
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.2	11.2	28.6
YOLOv9n	640	38.3	NA	NA	2.0	7.7
YOLOv9s	640	46.8	NA	NA	7.2	26.7
YOLOv11n	640	39.5	56.1	1.5	2.6	6.5
YOLOv11s	640	47.0	90.0	2.5	9.4	21.5

Hình 2.3 So sánh các phiên bản YOLO

[5] Bảng so sánh trên thể hiện hiệu năng của các phiên bản mô hình YOLO từ YOLOv5 đến YOLOv11, ở các biến thể nhẹ (n – nano) và nhỏ (s – small), với kích thước ảnh đầu vào cố định là 640 pixels. Các thông số đánh giá gồm: độ chính xác trung bình (mAP 50-95), tốc độ xử lý trên CPU, tốc độ xử lý trên GPU (TensorRT10), số lượng tham số rất lớn và độ phức tạp tính toán (FLOPs, tỷ phép toán).

Các phiên bản mới của dòng mô hình phát hiện đối tượng như YOLOv8, YOLOv9 và YOLOv11 đã có những cải tiến đáng kể về độ chính xác và tốc độ xử lý. Trong đó, YOLOv11s đạt mAP cao nhất (47.0), đồng thời duy trì tốc độ xử lý tốt và kích thước mô hình hợp lý (9.4 triệu tham số). So với các phiên bản cùng cấp trước đó, YOLOv11 thể hiện sự tối ưu hóa vượt trội về kiến trúc và hiệu quả tính toán. Đặc biệt, biến thể YOLOv11n cho thấy khả năng xử lý trên CPU rất ấn tượng (56.1 ms), với số lượng tham số và FLOPs thấp nhất – phù hợp cho các thiết bị có tài nguyên hạn chế như IoT hoặc điện thoại di động.

Mặc dù các phiên bản mới mang lại nhiều lợi ích, việc sử dụng YOLOv5 vẫn được duy trì trong một số trường hợp do các lý do sau:

- **Yêu cầu phần cứng cao hơn:** Các mô hình mới thường cần GPU mạnh hoặc môi trường tính toán hiện đại để đạt hiệu suất tối ưu, trong khi YOLOv5 có thể hoạt động ổn định trên hệ thống phổ thông.

- **Tính ổn định và cộng đồng hỗ trợ rộng rãi:** YOLOv5 đã được sử dụng phổ biến trong thời gian dài, có tài liệu đầy đủ và cộng đồng lớn, giúp dễ dàng triển khai và xử lý sự cố.
- **Tương thích với hệ thống hiện tại:** Việc chuyển sang phiên bản mới có thể đòi hỏi điều chỉnh pipeline xử lý, định dạng dữ liệu hoặc môi trường triển khai, gây khó khăn trong các dự án đang vận hành.

Tổng kết, các phiên bản mới như YOLOv8, YOLOv9 và YOLOv11 đạt được sự cân bằng tốt giữa độ chính xác và tốc độ. Đối với các ứng dụng yêu cầu mô hình nhẹ, các biến thể nano hoặc small của các phiên bản này là lựa chọn phù hợp. Tuy nhiên, YOLOv5 vẫn là giải pháp hiệu quả trong bối cảnh yêu cầu phần cứng hạn chế và cần sự ổn định trong triển khai.

2.1.4 Thành phần trong bài toán

Gồm hai model chính sẽ được training để nhận diện hai thành phần cần nhận diện trong ảnh là phát hiện khung biển số và nhận dạng các ký tự xuất hiện trên biển số xe:

Khối thứ nhất là khối phát hiện biển số xe. Dữ liệu đầu vào được thu thập từ video, camera hoặc các hình ảnh chụp các phương tiện giao thông có chứa biển số. Từ tập dữ liệu này, mô hình sẽ được huấn luyện để có khả năng phát hiện và khoanh vùng vị trí biển số trên ảnh hoặc video. Các dữ liệu ảnh đầu vào thường rất đa dạng, có thể chứa nhiều biển số xe trong cùng một ảnh, với sự khác biệt về góc chụp, tỷ lệ kích thước, độ sắc nét và mức độ che khuất.

Khối thứ hai là khối phân tách và nhận dạng ký tự. Từ vùng biển số đã được phát hiện ở khối thứ nhất, khối này sẽ thực hiện phân tách các ký tự trên biển số, sau đó nhận dạng từng ký tự và trả về kết quả là vùng biển số đã khoanh vùng kèm theo dãy ký tự nhận diện được.

Bài toán kết hợp chặt chẽ hai khối trên: đầu ra của khối thứ nhất (vùng chứa biển số) sẽ trở thành đầu vào cho khối thứ hai. Sau khi nhận diện ký tự, hệ thống sẽ xuất ra kết quả cuối cùng gồm vùng biển số đã phát hiện và chuỗi ký tự tương ứng.

Việc định vị biển số xe dựa vào các tính chất của biển số như: hình dạng, màu sắc của biển số so với nền. Biển số xe có hình chữ nhật với kích thước chiều

dài và rộng không thay đổi. Từ đó chúng ta có thể tìm các vùng có hình tứ giác trong hình, tính tỉ lệ giữa hai chiều của hình so với khoảng giá trị cho trước để xem thử vùng đó có chứa biển số hay không. Ngoài ra, định vị biển số còn dựa trên màu sắc của biển số. Hầu hết các biển số xe Việt Nam đều có thể nhận diện qua khung hình chữ nhật, nền trắng, chữ đen (loại trừ các loại biển số ít phổ biến như biển số quân đội, các cơ quan tình báo nhà nước). Dựa vào các đặc điểm phổ biến đó, ta có thể đào tạo được một mô hình có thể xác định được vùng chứa biển số.

2.2 Quá trình huấn luyện

2.2.1 Bộ dữ liệu

Trong mô hình nhận diện biển số xe này, em sử dụng hai bộ dữ liệu được thu thập từ các nguồn công khai [6]: một bộ dành cho bài toán phát hiện biển số và một bộ phục vụ cho việc nhận dạng ký tự trên biển số.

Cụ thể, bộ LP_detection bao gồm các hình ảnh phương tiện giao thông có biển số xe, được lưu trong thư mục Images. Phần nhãn của bộ dữ liệu, được lưu trong thư mục Labels dưới dạng các tệp .txt, ghi lại tọa độ các bounding box khoanh vùng vị trí biển số trong mỗi ảnh. Bộ dữ liệu này giúp huấn luyện mô hình có khả năng phát hiện chính xác vị trí biển số xe trong nhiều điều kiện khác nhau như thay đổi về góc chụp, độ sáng, hoặc che khuất một phần.

Bộ OCR tập hợp các hình ảnh biển số xe đã được cắt sẵn từ ảnh gốc. Thư mục Images của bộ này chứa các biển số xe riêng lẻ, còn thư mục Labels lưu thông tin khoanh vùng các ký tự trên từng biển số. Bộ dữ liệu này đóng vai trò huấn luyện mô hình nhận dạng ký tự, giúp mô hình có thể phân tách và nhận diện chính xác từng ký tự trên biển số xe, bất kể phong chữ, kiểu dáng hay tình trạng biển số.

Cấu trúc của cả hai bộ dữ liệu đều tuân theo chuẩn thông dụng, gồm hai thư mục chính là Images và Labels, mỗi thư mục lại được chia nhỏ thành hai tập: train và val, với tỷ lệ phân chia 80% cho tập huấn luyện (folder train) và 20% cho tập kiểm thử (folder val). Cách tổ chức này giúp đảm bảo quá trình huấn luyện có đủ dữ liệu đa dạng, đồng thời kiểm thử mô hình trên một tập dữ liệu độc lập để đánh giá độ chính xác và khả năng tổng quát của mô hình sau khi huấn luyện.

Với quá trình gán nhãn ta có thể sử dụng các công cụ tự động như Label Studio, labellmg, roboflow... giúp tự động hóa quá trình gán nhãn, rất hữu ích cho bộ dữ liệu ảnh lớn.

Nội dung được gán nhãn trong file txt sẽ theo cấu trúc sau, mỗi dòng trong file phải bao gồm các thông tin dưới đây:

- Class_index x_center y_center width height
- Giá trị class_index sẽ được nhập phụ thuộc vào số thứ tự của class đó được định nghĩa trong file yaml, bốn giá trị còn lại phải nằm trong khoảng từ [0:1], để đảm bảo điều đó thì ta sẽ chia các giá trị này với kích thước chiều dài và chiều rộng của bức ảnh đó [7].



Hình 2.4 Hình ảnh ví dụ cho quá trình đánh nhãn

Bảng 2.1 Nội dung của file txt

0	0.481719	0.634028	0.690625	0.713278
0	0.741094	0.524306	0.314750	0.933398
27	0.364844	0.795833	0.078125	0.400000

Ví dụ với hình ảnh Zidane.jpg và Zidane.txt

- Phát hiện ba vật thể chính trong bức ảnh, hai con người và một cái cà vạt (class index lần lượt là 0 và 27)
- Với người đầu tiên là Zidane thì tọa độ trung tâm của (x; y) = (0.481719; 0.634028), chiều rộng và chiều cao của nhân vật này trong bức ảnh lần lượt là 0.69625 và 0.713278

- Tương tự với đó là nhân vật thứ hai trong bức ảnh Ancelotti
- Vật thể thứ ba trong bức ảnh là chiếc cà vạt (class_index = 27), tọa độ trung tâm (x; y) = (0.364844; 0.795833), chiều rộng và chiều cao của vật thể này là 0.078125 và 0.400000.

2.2.2 Các chỉ số đánh giá hiệu suất mô hình

Trong quá trình xây dựng một mô hình Machine Learning, việc đánh giá mô hình là một bước không thể thiếu để xác định chất lượng và mức độ phù hợp của mô hình với bài toán cụ thể. Việc lựa chọn đúng thước đo đánh giá sẽ giúp ta đưa ra quyết định chính xác trong việc chọn mô hình tối ưu. Dưới đây là một số phương pháp và chỉ số đánh giá phổ biến [8]:

Ma trận nhầm lẫn cung cấp cái nhìn chi tiết về cách mô hình phân loại đúng hoặc sai các điểm dữ liệu. Các thành phần chính bao gồm:

- **True Positive (TP):** Dự đoán đúng lớp dương (positive).
- **True Negative (TN):** Dự đoán đúng lớp âm (negative).
- **False Positive (FP):** Dự đoán sai, thực tế là âm nhưng dự đoán là dương.
- **False Negative (FN):** Dự đoán sai, thực tế là dương nhưng dự đoán là âm.

Từ đó, ta có thể tính các chỉ số phụ:

- **True Positive Rate (TPR) hay Recall:** $\frac{TP}{TP + FN}$
- **False Positive Rate (FPR):** $\frac{FP}{FP + TN}$
- **True Negative Rate (TNR):** $\frac{TN}{TN + FP}$
- **False Negative Rate (FNR):** $\frac{FN}{FN + TP}$
- **Precision** Tỷ lệ dự đoán đúng trong số các dự đoán là positive: $\frac{TP}{TP + FP}$

Chỉ số mAP là chỉ số đánh giá phổ biến trong các bài toán **Object**

Detection. Để tính mAP, ta thực hiện các bước sau:

- Tính Precision và recall tại nhiều ngưỡng khác nhau cho từng lớp đối tượng
- Vẽ đường cong Precision-Recall (PR Curve) cho từng lớp
- Tính diện tích dưới đường cong đó (tính AP – Average Precision) cho mỗi lớp

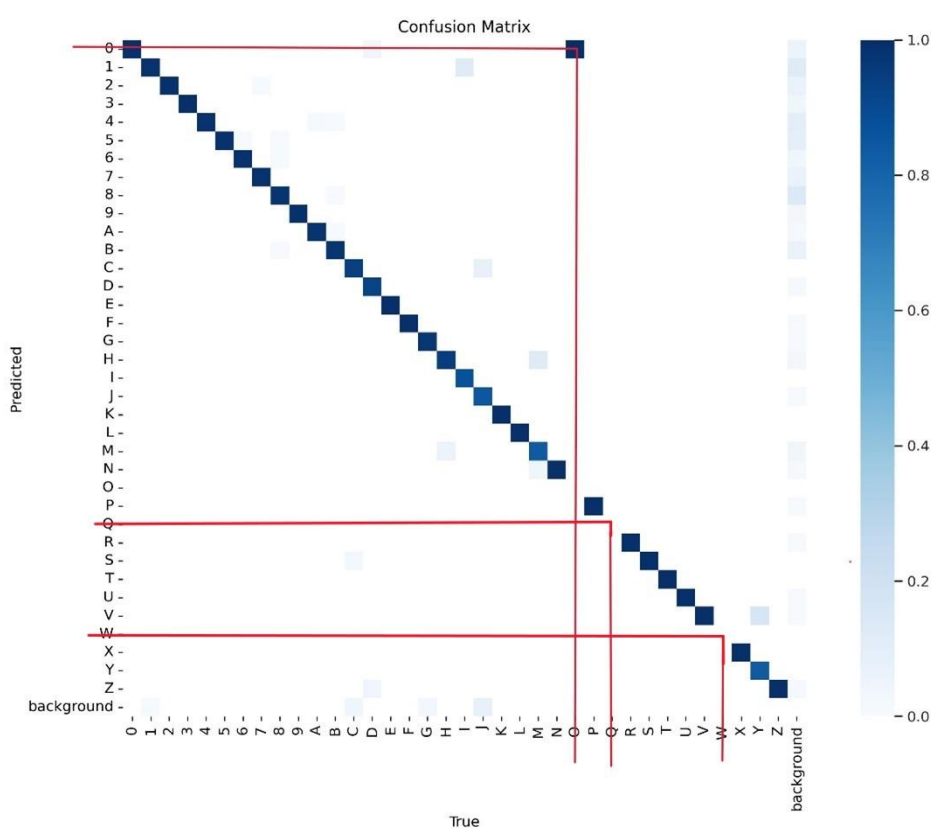
$$AP = \sum_{n=1}^N (R_n - R_{n-1}) \cdot P_n$$

Trong đó

- P_n và R_n là giá trị Precision và recall tại điểm thứ n trên PR Curve.
- PR Curve bắt đầu từ điểm (0, 1) và kết thúc tại (1, 0)

Cuối cùng để có được giá trị mAP thì chỉ cần tính trung bình

$$mAP = \frac{1}{K} \sum_{k=1}^K AP_k$$



Hình 2.5 Ma trận nhầm lẫn sau khi huấn luyện mô hình

Hình là kết quả thực tế thu được sau khi training model, dễ dàng nhận thấy rằng ký tự “O”, ký tự “Q” và “W” sẽ đưa ra các kết quả không khả quan, tuy nhiên đối chiếu với bảng tóm tắt dưới đây sẽ giải thích được lý do tại sao.

Fusing layers...

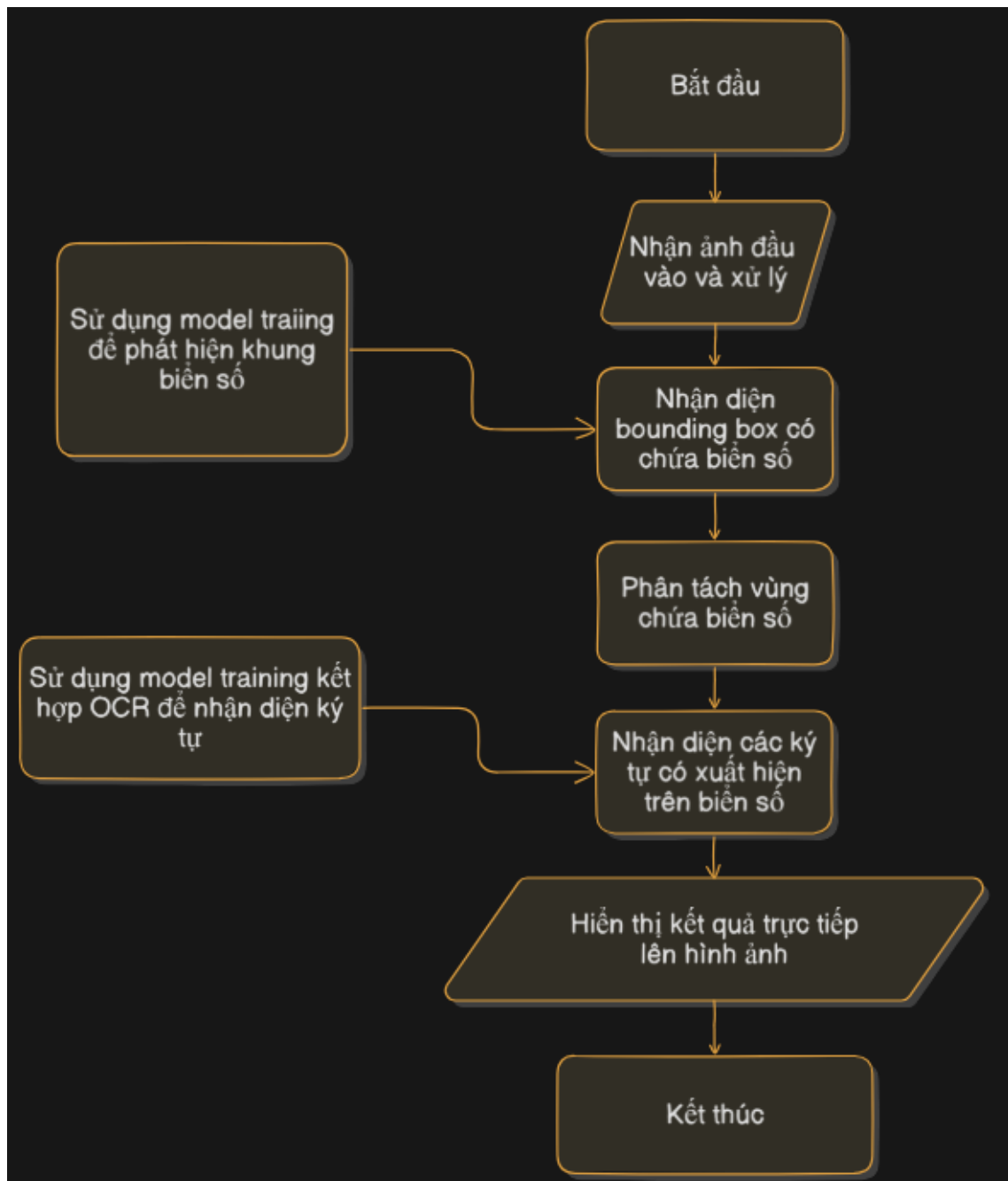
Model summary: 212 layers, 20994369 parameters, 0 gradients, 48.3 GFLOPs

Class	Images	Instances	P	R	mAP50	mAP50-95: 100% 10/10 [00:10<00:00, 1.00s/it]
all	632	5108	0.979	0.94	0.946	0.704
0	632	364	0.991	0.995	0.99	0.763
1	632	690	0.987	0.973	0.981	0.619
2	632	409	0.995	0.993	0.994	0.752
3	632	324	1	0.992	0.995	0.764
4	632	357	0.99	0.989	0.992	0.718
5	632	670	0.985	0.992	0.99	0.776
6	632	348	0.983	0.987	0.99	0.755
7	632	340	0.985	0.989	0.988	0.743
8	632	312	0.987	0.983	0.99	0.756
9	632	479	0.994	0.994	0.99	0.757
A	632	55	0.973	0.982	0.993	0.709
B	632	143	0.972	0.993	0.992	0.798
C	632	51	1	0.93	0.964	0.774
D	632	26	0.924	0.933	0.94	0.658
E	632	19	0.992	1	0.995	0.73
F	632	67	0.983	1	0.991	0.763
G	632	67	0.997	0.97	0.983	0.725
H	632	35	0.962	0.886	0.973	0.786
I	632	8	0.988	0.875	0.877	0.402
J	632	13	0.986	0.846	0.865	0.683
K	632	28	0.993	1	0.995	0.737
L	632	37	0.996	1	0.995	0.756
M	632	24	0.871	0.844	0.951	0.689
N	632	16	0.995	1	0.995	0.73
O	632	1	1	0	0.00173	0.00139
P	632	34	0.989	1	0.995	0.756
R	632	4	0.976	1	0.995	0.821
S	632	39	0.972	1	0.994	0.719
T	632	33	0.996	1	0.995	0.808
U	632	26	0.958	1	0.995	0.734
V	632	12	0.904	1	0.937	0.676
X	632	17	1	0.989	0.995	0.598
Y	632	6	0.975	0.833	0.845	0.69
Z	632	54	0.991	1	0.995	0.804

Hình 2.6 Kết quả nhận diện các ký tự xuất hiện trong bộ dữ liệu

- Trong danh sách kết quả, ký tự “Q” và “W” không xuất hiện trong bộ dữ liệu huấn luyện, dẫn đến việc mô hình không thể học và dự đoán các ký tự này.
- Ký tự “O”: Chỉ xuất hiện một lần trong bộ dữ liệu và mô hình đã dự đoán sai, điều này được thể hiện rõ trong ma trận nhầm lẫn.

⇒ **Nhận xét:** Trong thực tế, ký tự “O” không được sử dụng trên biển số xe ở Việt Nam do dễ bị nhầm lẫn với số 0. Do đó, sai số liên quan đến ký tự “O” có thể được bỏ qua mà không ảnh hưởng đến hiệu quả của mô hình trong các ứng dụng thực tế.



Hình 2.7 Sơ đồ hoạt động của mô hình

2.2.3 Kết quả mô hình nhận diện biển số

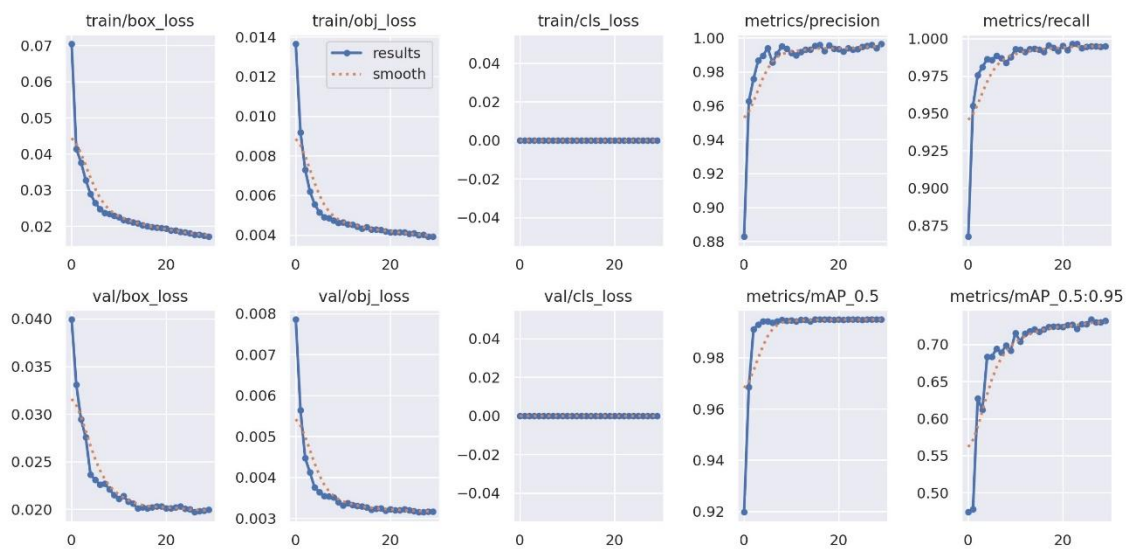
Sử dụng phiên bản free-to-use của Google Colab, thực hiện việc clone thư viện yolov5 trên github để bắt đầu quá trình training model.

Bảng 2.2 Đoạn lệnh huấn luyện mô hình nhận diện biển số

```
!python train.py --img 320 --batch 32 --epochs 30 --data /content/yolov5/data/LP_detection.yaml --weights yolov5m.pt --cache
```

Mô hình được huấn luyện với file train.py của thư viện YOLOv5 với các tham số được tối ưu cho bài toán nhận diện biển số xe. Cụ thể, ảnh đầu vào được resize

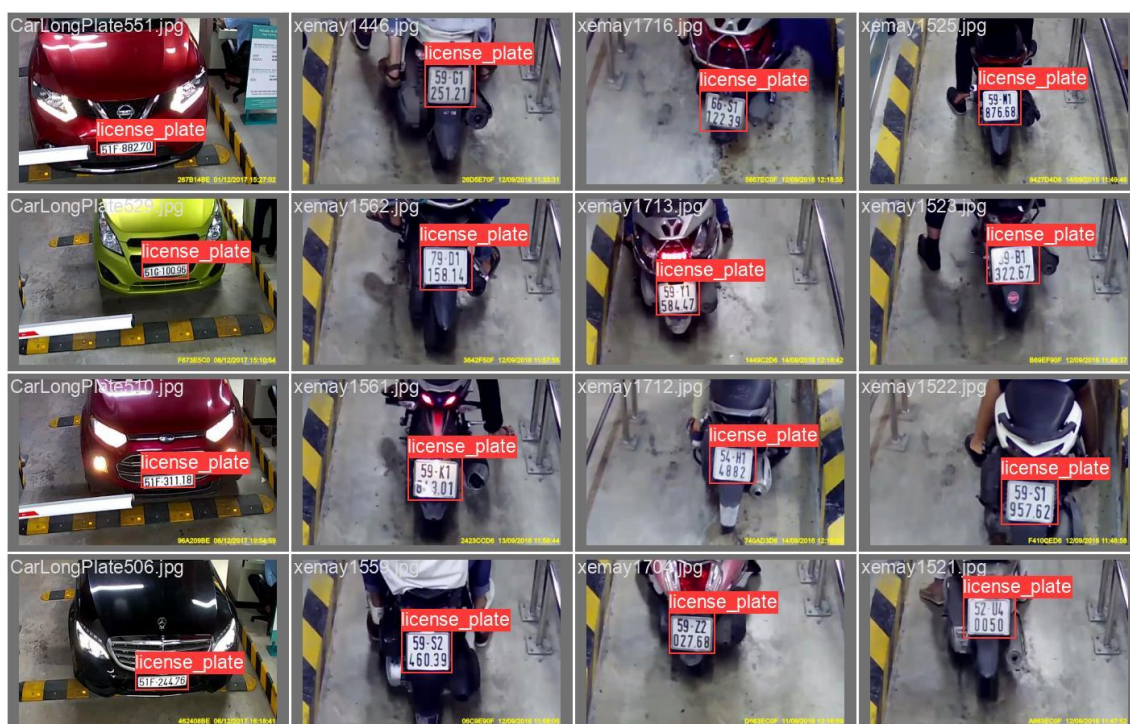
về kích thước 320×320-pixel nhằm giảm chi phí tính toán và tăng tốc độ xử lý. Batch size được đặt là 32, cho phép mô hình cập nhật trọng số sau mỗi 32 ảnh, giúp cân bằng giữa tốc độ huấn luyện và độ ổn định. Quá trình huấn luyện diễn ra trong 30 epochs, đủ để mô hình học được các đặc trưng cần thiết từ tập dữ liệu gồm 6607 ảnh. File cấu hình dữ liệu LP_detection.yaml định nghĩa các lớp đối tượng và đường dẫn đến tập huấn luyện và xác thực. Mô hình khởi tạo từ trọng số pretrained của YOLOv5m (yolov5m.pt), giúp rút ngắn thời gian huấn luyện nhờ kế thừa các đặc trưng đã học từ tập dữ liệu lớn. Tham số --cache được sử dụng để nạp toàn bộ dữ liệu vào bộ nhớ RAM, từ đó tăng tốc độ đọc ảnh và cải thiện hiệu suất huấn luyện.



Hình 2.8 Các đồ thị trong quá trình huấn luyện mô hình nhận diện biển số

- **Box loss & Objectness loss (train/val):** Cả hai loại mất mát đều giảm đều theo thời gian, cho thấy mô hình học được cách phân biệt các biển số xe xuất hiện trong hình.
- **Classification loss:** Gần như bằng 0 trong cả huấn luyện và xác thực → điều này đạt được do trong mô hình chỉ có một vật thể duy nhất cần phát hiện đó là biển số xe (class index = 0).
- **Precision & Recall:** Cả hai chỉ số đều tăng và ổn định gần mức 1.0, cho thấy mô hình rất chính xác và bao phủ tốt các đối tượng cần phát hiện.
- **mAP@0.5:** tỷ lệ phần trăm gần đạt 100 phần trăm, phản ánh mô hình dự đoán rất chính xác ở ngưỡng IoU = 0.5.

- **mAP@0.5:0.95**: Giá trị trung bình ổn định lớn hơn 0.7, cho thấy mô hình vẫn giữ được hiệu suất cao ngay cả ở các ngưỡng IoU khắt khe hơn.



Hình 2.9 Kết quả huấn luyện mô hình nhận diện biển số

Bảng 2.3 Tóm tắt kết quả quá trình huấn luyện mô hình nhận diện biển số

Thông số	Thời gian training	Số lượng hình ảnh	Độ chính xác	Số epochs
Giá trị	35 phút	6607	99.5%	30

2.2.4 Kết quả mô hình nhận diện ký tự

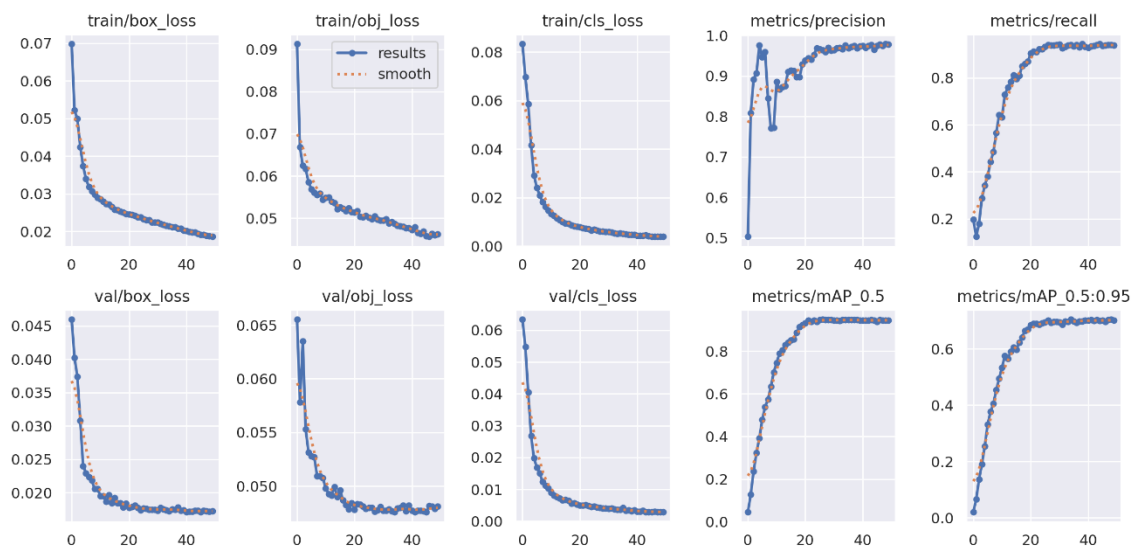
Bảng 2.4 Đoạn lệnh huấn luyện mô hình nhận diện ký tự

```
!python train.py --img 640 --batch 32 --epochs 50 --data
/content/yolov5/data/Letter_detect.yaml --weights yolov5m.pt --cache
```

Tương tự với quá trình training model để nhận diện ký tự, sử dụng lệnh huấn luyện với các tham số cụ thể như sau: kích thước ảnh đầu vào được thiết lập là 640x640 pixel nhằm đảm bảo sự cân bằng giữa tốc độ xử lý và độ chính xác của mô hình. Kích thước batch được đặt là 32, tức là mỗi lần huấn luyện mô hình sẽ xử lý 32 ảnh cùng lúc, phù hợp với khả năng bộ nhớ của hệ thống.

Tuy nhiên với số vòng lặp huấn luyện (epochs) sẽ được tăng lên 50 để mô hình có thể nhận diện tốt các ký tự. Tập dữ liệu được định nghĩa thông qua file YAML có đường dẫn /content/yolov5/data/Letter_detect.yaml, trong đó chứa

thông tin về đường dẫn ảnh, nhãn và số lượng lớp cần nhận diện. Trọng số khởi tạo được sử dụng là từ mô hình YOLOv5m đã được huấn luyện trước (yolov5m.pt), giúp tăng tốc độ hội tụ và cải thiện hiệu suất mô hình. Cuối cùng, tham số --cache được thêm vào để lưu trữ dữ liệu vào bộ nhớ đệm, giúp tăng tốc độ đọc dữ liệu trong quá trình huấn luyện. Việc lựa chọn các tham số này nhằm tối ưu hóa hiệu suất huấn luyện và đảm bảo mô hình đạt được độ chính xác cao trong nhiệm vụ nhận diện ký tự.



Hình 2.10 Các đồ thị trong quá trình huấn luyện mô hình nhận diện ký tự

- **train/box_loss**: Độ mất mát khi dự đoán bounding box trên tập huấn luyện giảm dần theo từng epoch, cho thấy mô hình ngày càng định vị vật thể chính xác hơn.
- **train/obj_loss**: Độ mất mát khi xác định sự tồn tại của vật thể cũng giảm đều, chứng tỏ mô hình học tốt trong việc phân biệt vật thể và nền.
- **train/cls_loss**: Độ mất mát phân loại trên tập huấn luyện giảm ổn định, phản ánh khả năng phân loại vật thể của mô hình được cải thiện.
- **val/box_loss**: Độ mất mát bounding box trên tập kiểm tra giảm tương tự như tập huấn luyện, cho thấy mô hình không bị overfitting và có khả năng tổng quát tốt.
- **val/obj_loss**: Độ mất mát objectness trên tập kiểm tra giảm đều, chứng minh mô hình duy trì hiệu suất phát hiện vật thể trên dữ liệu mới.

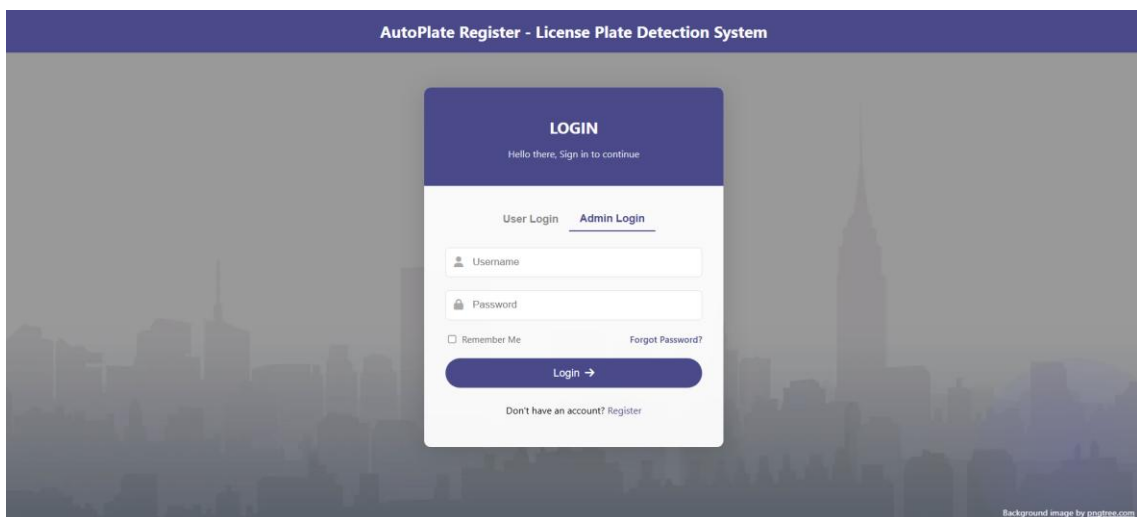
Bảng 2.5 Tóm tắt kết quả quá trình huấn luyện mô hình nhận diện ký tự

Thông số	Thời gian training	Số lượng hình ảnh	Độ chính xác	Số epochs
Giá trị	52 phút	2533	94.6%	50

2.3 TÍCH HỢP MÔ HÌNH VÀO HỆ THỐNG

2.3.1 Mô tả tổng quan

Dựa trên kết quả từ mô hình đã được huấn luyện, hệ thống nhận diện và đăng ký biển số xe được xây dựng bằng ngôn ngữ lập trình Python, sử dụng Flask cho phần backend. Cơ sở dữ liệu MySQL được tích hợp để quản lý thông tin người dùng, dữ liệu biển số và trạng thái xử lý các đơn đăng ký. Việc kết hợp giữa mô hình học sâu và nền tảng web giúp hệ thống hoạt động tự động, hỗ trợ người dùng một cách hiệu quả trong việc đăng ký và quản lý biển số xe.



Hình 2.12 Giao diện đăng nhập chính của hệ thống

Thông qua quá trình phát triển hệ thống, em đã từng bước triển khai các thành phần chính như nhận diện biển số từ hình ảnh, trích xuất ký tự, quản lý người dùng và xử lý quy trình phê duyệt qua giao diện web, đảm bảo tính thống nhất và khả năng mở rộng trong ứng dụng thực tiễn.



Hình 2.13 Sơ đồ sơ lược quy trình hoạt động của hệ thống

Sơ đồ trên minh họa cấu trúc chức năng chính của hệ thống quản lý đăng ký biển số xe dựa trên hai loại tài khoản: **tài khoản người dùng** và **tài khoản quản trị viên**, được truy cập sau bước đăng nhập.

Sau khi người dùng đăng nhập vào hệ thống, họ sẽ được phân loại quyền truy cập tương ứng:

2.3.2 Kết nối với ESP32-CAM

```

def gen_frames():
    global latest_plate
    # Load models once here
    yolo_LP_detect = torch.hub.load('yolov5', 'custom',
    path='model/LP_detector.pt', force_reload=True, source='local')
    yolo_license_plate = torch.hub.load('yolov5', 'custom',
    path='model/LP_ocr.pt', force_reload=True, source='local')
    yolo_license_plate.conf = 0.60
    stream = requests.get(ESP32_CAM_URL, stream=True, timeout=5)
    byte_data = b''
    frame_count = 0
    frame_skip = 2
    max_plates = 3
    while True:
        try:
            for chunk in stream.iter_content(chunk_size=2048):
                byte_data += chunk
                a = byte_data.find(b'\xff\xd8') # JPEG start
                b = byte_data.find(b'\xff\xd9') # JPEG end
                if a != -1 and b != -1 and b > a:

```

```

        jpg = byte_data[a:b + 2]
        byte_data = byte_data[b + 2:]
        frame = cv2.imdecode(np.frombuffer(jpg,
dtype=np.uint8), cv2.IMREAD_COLOR)
        if frame is None:
            continue
        frame = cv2.resize(frame, (640, 480),
interpolation=cv2.INTER_AREA)
        frame_count += 1
        if frame_count % frame_skip != 0:
            ret, buffer = cv2.imencode('.jpg', frame,
[int(cv2.IMWRITE_JPEG_QUALITY), 80])
            if not ret:
                continue
            frame_bytes = buffer.tobytes()
            yield (b'--frame\r\n'
                    b'Content-Type: image/jpeg\r\n\r\n' +
frame_bytes + b'\r\n')
            continue
        # Detect plates
        plates = yolo_LP_detect(frame, size=320)
        list_plates =
plates.pandas().xyxy[0].values.tolist()[:max_plates]
        for plate in list_plates:
            x, y, w, h = int(plate[0]), int(plate[1]),
int(plate[2] - plate[0]), int(plate[3] - plate[1])
            crop_img = frame[y:y + h, x:x + w]
            cv2.rectangle(frame, (x, y), (x + w, y + h),
(0, 0, 225), 2)

            for cc in range(2):
                for ct in range(2):
                    lp =
helper.read_plate(yolo_license_plate, utils_rotate.deskew(crop_img,
cc, ct))

                    if lp != "unknown":
                        latest_plate = lp
                        print(f"[STREAM DETECTED]
{latest_plate}")

                        cv2.putText(frame, lp, (x, y -
10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (36, 255, 12), 2)
                        break
            ret, buffer = cv2.imencode('.jpg', frame,
[int(cv2.IMWRITE_JPEG_QUALITY), 80])
            if not ret:
                continue
            frame_bytes = buffer.tobytes()
            yield (b'--frame\r\n'
                    b'Content-Type: image/jpeg\r\n\r\n' +
frame_bytes + b'\r\n')
        except (requests.exceptions.RequestException,
ConnectionError):
            print("Stream interrupted, attempting to reconnect...")
            stream = requests.get(ESP32_CAM_URL, stream=True,
timeout=5)

```

```
byte_data = b''  
continue
```

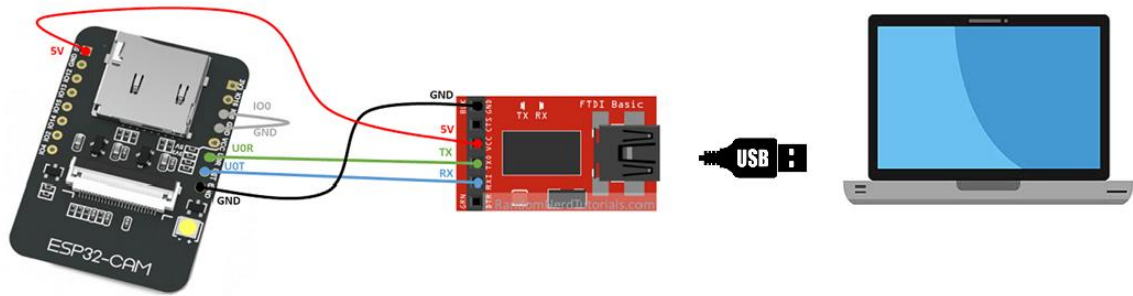
Đầu tiên, hàm `gen_frames()` được sẽ chạy liên tục, lấy dữ liệu hình ảnh từ một camera ESP32-CAM thông qua giao thức HTTP (ESP32-CAM được kết nối với wifi trong quá trình hoạt động). Camera này phát luồng video dưới dạng MJPEG, tức là một chuỗi các ảnh JPEG được gửi liên tục. Để kết nối đến luồng này, đoạn mã sử dụng thư viện `requests` với chế độ `stream = True`, cho phép đọc dữ liệu theo từng phần nhỏ (chunk) thay vì phải đợi toàn bộ nội dung được tải về. Điều này rất quan trọng khi làm việc với video trực tuyến, vì nó giúp xử lý từng khung hình ngay khi nhận được, giảm độ trễ.

Sau khi kết nối thành công, chương trình bắt đầu đọc dữ liệu nhị phân từ luồng. Nó tìm kiếm các byte đặc trưng đánh dấu điểm bắt đầu (`0xFFD8`) và kết thúc (`0xFFD9`) của một ảnh JPEG. Khi tìm thấy một ảnh đầy đủ, nó giải mã ảnh đó thành một mảng ảnh OpenCV để xử lý tiếp. Nếu ảnh hợp lệ, nó sẽ được resize về kích thước chuẩn (`640x480`) để đảm bảo hiệu suất và độ chính xác khi đưa vào mô hình.

Tiếp theo là phần quan trọng: tải và sử dụng mô hình đã huấn luyện. Ở đầu hàm, hai mô hình YOLOv5 được tải bằng `torch.hub.load`. Đây là cách PyTorch tải các mô hình từ kho lưu trữ hoặc từ file cục bộ. Trong trường hợp này, đang sử dụng mô hình tùy chỉnh đã được huấn luyện trước đó và lưu dưới dạng file `.pt`. Mô hình đầu tiên (`LP_detector.pt`) dùng để phát hiện vị trí biển số xe trong ảnh. Mô hình thứ hai (`LP_ocr.pt`) dùng để nhận dạng ký tự trên biển số sau khi đã cắt và xử lý ảnh biển số.

Sau khi mô hình phát hiện được vùng chứa biển số, chương trình sẽ cắt vùng đó ra khỏi ảnh gốc, sau đó thử nhiều góc xoay khác nhau để khử nghiêng (deskew) nhằm tăng độ chính xác khi nhận dạng. Mỗi vùng biển số được đưa vào mô hình OCR để đọc ký tự. Nếu mô hình trả về kết quả hợp lệ (khác "unknown"), biển số sẽ được lưu lại và hiển thị lên ảnh bằng OpenCV.

Cuối cùng, ảnh đã xử lý sẽ được mã hóa lại thành JPEG và trả về dưới dạng luồng MJPEG để hiển thị trên trình duyệt hoặc giao diện người dùng.

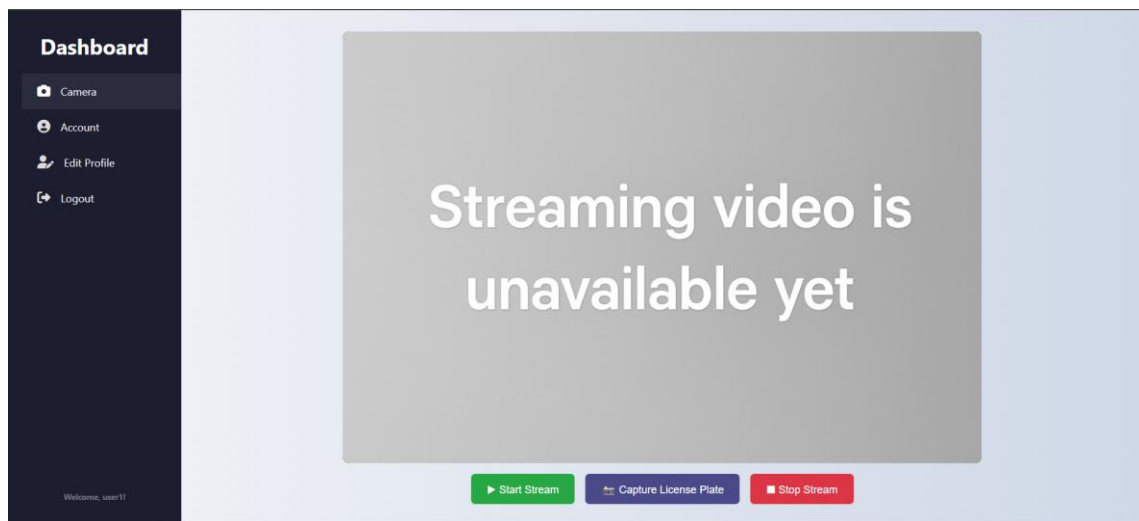


Hình 2.14 Sơ đồ kết nối ESP32-CAM với mạch chuyển FTDI

2.3.3 Tài khoản người dùng

Với **tài khoản người dùng**, hệ thống cho phép thực hiện ba chức năng chính:

- Kết nối tới thiết bị livestream ESP32-CAM để nhận diện biển số tự động, sau đó gửi đơn đăng ký tới cho quản trị viên.
- Theo dõi trạng thái tài khoản và đơn đăng ký biển số xe mà tài khoản đã gửi.
- Chỉnh sửa thông tin cá nhân trong tài khoản, chẳng hạn như họ tên, email, hoặc các dữ liệu liên quan khác.



Hình 2.15 Giao diện chính của người dùng

2.3.4 Tài khoản quản trị viên

Với **tài khoản quản trị viên**, hệ thống cung cấp ba chức năng chuyên biệt nhằm phục vụ công tác phê duyệt và giám sát:

- Xem danh sách đơn đăng ký biển số xe được gửi từ các người dùng.

- Phê duyệt hoặc từ chối (phải điền rõ lý do từ chối) từng đơn đăng ký thông qua giao diện quản lý. Quản trị viên có thể xem hình ảnh được chụp bởi người dùng kèm theo biển số xe được nhận diện tự động bởi hệ thống để đưa ra quyết định.
- Theo dõi lịch sử xử lý đơn của tất cả các quản trị viên, giúp đảm bảo tính minh bạch và giám sát quá trình xét duyệt.

Admin Dashboard

PLATE REGISTRATIONS

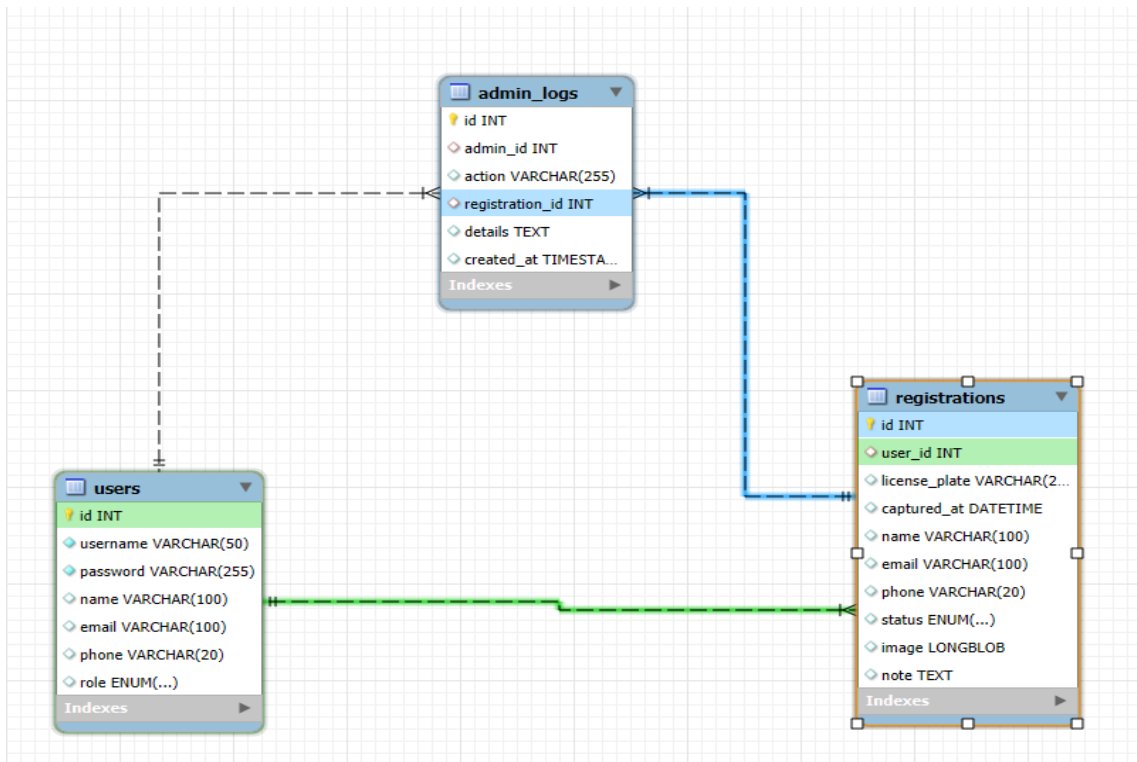
Search by License Plate... [Export to Excel](#)

LICENSE PLATE	CAPTURED AT	NAME	EMAIL	PHONE	STATUS	ADMIN NOTE	CAPTURED IMAGE	ACTIONS
59N3-34388	2025-06-22 10:46:47	user1	agdsfds@gmail.com	0147852	PENDING	—		Approve Deny
59N3-34388	2025-05-18 11:13:56	user1	agdsfds@gmail.com	092922222	APPROVED	—		approved
29C1-99999	2025-05-18 11:13:19	user1	agdsfds@gmail.com	092922222	APPROVED	—		approved
36A-88888	2025-05-18 11:12:55	user1	agdsfds@gmail.com	092922222	DENIED	biển số không hợp lệ ví dụ		denied
6389-99999	2025-05-18 11:12:38	user1	agdsfds@gmail.com	092922222	APPROVED	—		approved
59N3-34388	2025-05-18 11:12:16	user1	agdsfds@gmail.com	092922222	APPROVED	—		approved

Welcome, minhtrixadmin!

Hình 2.16 Giao diện chính của quản trị viên

2.3.5 Cơ sở dữ liệu



Hình 2.17 Mô hình ERD của cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống nhận diện và đăng ký biển số xe được đặt tên là `vehicle_system`, thiết kế theo mô hình quan hệ, bao gồm ba bảng chính: `users`, `registrations` và `admin_logs`. Mỗi bảng đảm nhận một chức năng riêng biệt nhưng có mối liên kết chặt chẽ nhằm đảm bảo khả năng lưu trữ, truy xuất và quản lý thông tin hiệu quả. Trong đó, bảng `users` quản lý thông tin người dùng, bảng `registrations` lưu trữ các đơn đăng ký biển số xe và bảng `admin_logs` ghi lại toàn bộ lịch sử hành động của quản trị viên.

Bảng 2.6 Lớp dữ liệu của người dùng

Bảng users		
Trường thông tin	Kiểu dữ liệu	Mô tả
Id	INT (primary key)	Khóa chính, dùng để định danh tài khoản
Username	VARCHAR(50)	Tên đăng nhập của tài khoản
Password	VARCHAR(255)	Mật khẩu người dùng, được mã hóa theo chuẩn để nâng cao bảo mật
Name	VARCHAR(100)	Địa chỉ email của tài khoản
Phone	VARCHAR(20)	Số điện thoại được liên kết với tài khoản
Role	ENUM('user', 'admin')	Phân quyền của tài khoản, người dùng hoặc quản trị viên

Bảng 2.7 Lớp dữ liệu của đơn đăng ký

Bảng registrations		
Trường thông tin	Kiểu dữ liệu	Mô tả
id	INT (primary key)	Đóng vai trò khóa chính, là mã nhận diện của đơn đăng ký mà người dùng gửi
users_id	INT (foreign key)	Khóa phụ, là mã nhận diện tài khoản của người dùng
license_plate	VARCHAR(20)	Chứa ký tự được phát hiện trên biển số xe
captured_at	DATETIME	Ghi nhận thời điểm mà đơn đăng ký được gửi đi
name	VARCHAR(100)	Tên tài khoản người dùng
email	VARCHAR(100)	Email của người dùng
phone	VARCHAR(20)	Số điện thoại của người dùng
status	ENUM(...)	Trạng thái của đơn đăng ký, chờ xử lý, đã được duyệt hoặc bị từ chối

image	LONGBLOB	Dùng để lưu lại hình ảnh phát hiện biển số tự động
note	TEXT	Ghi chú (trong trường hợp đơn bị từ chối, quản trị viên phải ghi rõ lý do từ chối)

Bảng 2.8 Lớp dữ liệu của quản trị viên

Bảng admin_logs		
Trường thông tin	Kiểu dữ liệu	Mô tả
id	INT (Primary Key)	Khóa chính,
admin_id	INT (Foreign Key)	Mã xác thực tài khoản quản trị viên thực hiện
action	VARCHAR(255)	Quyết định của quản trị viên với đơn đăng ký (duyet hoặc từ chối)
registration_id	INT(Foreign Key)	Mã nhận diện của đơn đăng ký
details	TEXT	Thông tin chi tiết về hành động của quản trị viên
created_at	TIMESTAMP	Thời điểm mà quản trị viên duyệt đơn

Mối quan hệ giữa các bảng:

- **users ↔ registrations**
 - Mối quan hệ: Một-nhiều (1:N)
 - Diễn giải: Một người dùng có thể gửi nhiều đơn đăng ký biển số xe.
 - Liên kết: registrations.user_id là khóa ngoại tham chiếu đến users.id.
- **users ↔ admin_logs**
 - Mối quan hệ: Một-nhiều (1:N)
 - Diễn giải: Một tài khoản quản trị viên có thể thực hiện nhiều hành động xử lý đơn đăng ký của người dùng.
 - Liên kết: admin_logs.admin_id là khóa ngoại tham chiếu đến users.id.
- **registrations ↔ admin_logs**
 - Mối quan hệ: Một-nhiều (1:N)
 - Diễn giải: Mỗi đơn đăng ký có thể được xử lý bởi bất kỳ quản trị viên nào, sau khi duyệt thì đơn sẽ về trạng thái không thể xử lý được nữa.

- Liên kết: `admin_logs.registration_id` là khóa ngoại tham chiếu đến `registrations.id`.

Phần 3: KẾT QUẢ VÀ ĐÁNH GIÁ

3.1 Kết quả thực tế

3.1.1 Nhận diện hình ảnh

STT	Hình ảnh	Ghi chú
1		Kết quả: 59N3-3488 , chính xác nhưng khung nhận diện chưa bao quát đúng do biển số để quá nghiêng.
2		Kết quả: 59N3-34388 , kết quả chính xác với khung nhận diện biển số bao quát đúng, mặc dù điều kiện không lý tưởng (ảnh mờ, biển số xe để xa khung hình).
3		Kết quả: 70F1-66666 , kết quả chính xác, khung nhận diện biển số bao quát đủ mặc dù biển số để nghiêng
4		Kết quả: 29C1-99999 , kết quả chính xác, khung bao quát đủ trong khi biển số để bị hơi lệch theo trục ngang.

5	 A license plate with the text '59N3-34388'. The top part '59-N3' is on a light background, and the bottom part '343.88' is on a darker background. A red rectangular box highlights the entire plate. Above the plate, the text '59N3-34388' is written in green.	Kết quả: 59N3-34388 , kết quả chính xác, khung bao quát nhận diện đúng biển số, điều kiện lý tưởng, các ký tự rõ ràng sắc nét.
6	 A license plate with the text '30K-888.18'. The top part '30K-888' is on a light background, and the bottom part '.18' is on a darker background. A red rectangular box highlights the entire plate. Above the plate, the text '30K88818' is written in green.	Kết quả: 30K88818 , kết quả chính xác, khung bao quát nhận diện đúng biển số, điều kiện lý tưởng, các ký tự rõ ràng sắc nét, tuy nhiên sẽ phải giữ camera cố định lâu hơn so với biển số xe máy (có hai hàng ký tự).

3.1.2 Hoạt động hệ thống

Qua quá trình kiểm thử và vận hành, hệ thống cho thấy khả năng hoạt động ổn định và đạt hiệu suất như mong đợi. Tính năng nhận diện biển số xe hoạt động chính xác, cho kết quả phù hợp với mục tiêu đề ra. Hệ thống phản hồi nhanh, xử lý hình ảnh hiệu quả và cho ra kết quả nhận diện đúng trong đa số trường hợp thử nghiệm.

Các chức năng tương tác giữa người dùng và quản trị viên thông qua đơn đăng ký biển số được thực hiện trơn tru, không xảy ra lỗi trong quá trình nhập, gửi hoặc duyệt dữ liệu. Giao diện người dùng thân thiện, dễ thao tác, hỗ trợ tốt cho cả hai vai trò.

Cơ sở dữ liệu vận hành ổn định, lưu trữ đầy đủ thông tin người dùng, đơn đăng ký và hình ảnh đã chụp từ camera. Việc truy xuất dữ liệu diễn ra nhanh chóng, không có hiện tượng trễ hoặc mất dữ liệu. Nhìn chung, hệ thống đáp ứng tốt các yêu cầu kỹ thuật và tính năng ban đầu, thể hiện khả năng ứng dụng trong thực tế.

3.2 Đánh giá và nhận xét

3.2.1 Tính ứng dụng thực tiễn

Đồ án này mang lại nhiều giá trị thực tiễn trong bối cảnh nhu cầu quản lý phương tiện ngày càng tăng cao tại các khu dân cư, bãi giữ xe, tòa nhà văn phòng và khu công nghiệp.

Việc sử dụng ESP32-CAM – một thiết bị nhỏ gọn, chi phí thấp nhưng tích hợp camera và khả năng kết nối Wi-Fi – giúp hệ thống trở nên linh hoạt, dễ triển khai và tiết kiệm chi phí so với các giải pháp truyền thống.

Hệ thống có thể tự động nhận diện biển số xe khi phương tiện đi qua, từ đó hỗ trợ việc đăng ký, lưu trữ và kiểm soát ra vào một cách chính xác và nhanh chóng. Điều này giúp giảm thiểu sự phụ thuộc vào con người, hạn chế sai sót trong quá trình ghi nhận thông tin, đồng thời nâng cao hiệu quả vận hành. Ngoài ra, dữ liệu biển số có thể được tích hợp với các hệ thống quản lý khác như kiểm soát an ninh, thanh toán tự động hoặc thống kê lưu lượng xe ra vào.

Với khả năng hoạt động thời gian thực và tính mở rộng cao, đề án này hoàn toàn có thể được ứng dụng trong các mô hình bãi giữ xe thông minh, khu đô thị thông minh (smart city), hoặc làm nền tảng cho các hệ thống giám sát giao thông hiện đại trong tương lai.

3.2.2 Đề xuất hướng cải tiến

Các phương thức cải thiện hiệu suất **về phần mềm**:

- Tách riêng việc tải mô hình AI: Thay vì tải mô hình YOLOv5 mỗi lần gọi hàm xử lý ảnh, bạn nên tải mô hình một lần duy nhất khi khởi động hệ thống. Điều này giúp giảm thời gian khởi tạo và tránh lặp lại thao tác nặng.
- Giảm độ phân giải ảnh đầu vào: Resize ảnh về kích thước nhỏ hơn (ví dụ 320x320) để giảm thời gian xử lý mà vẫn giữ được độ chính xác tương đối.
- Giảm tần suất xử lý khung hình: Không cần xử lý mọi khung hình từ camera. Có thể bỏ qua một số khung hình (ví dụ xử lý mỗi 3 khung hình) để giảm tải cho CPU.
- Chuyển mô hình sang định dạng tối ưu: Sử dụng định dạng ONNX hoặc TensorRT để tăng tốc độ suy luận, đặc biệt khi chạy trên thiết bị có hỗ trợ tăng tốc phần cứng.
- Xử lý ảnh song song: Dùng đa luồng hoặc hàng đợi để xử lý ảnh và nhận diện biển số song song với việc nhận luồng video, giúp hệ thống phản hồi nhanh hơn.
- Lọc và tránh xử lý trùng lặp: Lưu lại biển số đã nhận diện trong thời gian ngắn để tránh xử lý lại cùng một xe nhiều lần liên tiếp.

- Chuẩn bị bộ dữ liệu huấn luyện nhiều hơn và đánh nhãn một cách chi tiết và chính xác hơn.

Tối ưu phần cứng giúp hệ thống hoạt động ổn định hơn, đặc biệt trong môi trường thực tế:

- Sử dụng thiết bị xử lý mạnh hơn: Thay vì xử lý AI trên máy tính thông thường, có thể dùng các thiết bị chuyên dụng như Raspberry Pi 4, NVIDIA Jetson Nano hoặc Google Coral để tăng tốc độ xử lý mô hình.
- Tối ưu kết nối mạng: Đảm bảo ESP32-CAM và máy chủ xử lý nằm trong cùng mạng LAN để giảm độ trễ truyền tải video.
- Tăng cường nguồn cấp và tản nhiệt: Đảm bảo ESP32-CAM có nguồn điện ổn định và không bị quá nhiệt khi hoạt động liên tục.

TÀI LIỆU THAM KHẢO

- [1] *Module ESP32-CAM Bluetooth Camera OV2640*. [Online]. Available: <https://www.thegioiic.com/module-esp32-cam-bluetooth-camera-ov2640>
- [2] *ESP32-CAM Pinout Reference*. Last Minute Engineers. [Online]. Available: <https://lastminuteengineers.com/esp32-cam-pinout-reference/>
- [3] *Ultralytics Documentation – Train YOLOv5 on Custom Data*. [Online]. Available: https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/
- [4] Hussain, M., *YOLO-v1 to YOLO-v8: The Rise of YOLO and Its Complementary Nature Toward Digital Manufacturing and Industrial Defect Detection*, 2024.
- [5] Rao, S. N., *YOLOv11 Architecture Explained: Next-Level Object Detection with Enhanced Speed and Accuracy*, 2024.
- [6] *Vietnam License Plate Dataset*. Duy Dieu Nguyen, Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/duydieunguyen/licenseplates>
- [7] *Ultralytics Documentation – How to Train Your Data*. [Online]. Available: https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/
- [8] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi: *You Only Look Once: Unified, Real-Time Object Detection*.